ECE532 Project

ShoutMaster3000: Audio Controlled Dino Game

Group 23:
Sherry Li
Jasmine Huang
Haodong Ma

Supervising TA:
Adham Ragab

April 2025

Contents

1 Overview **1**
- 1.1 Background . . . 1
- 1.2 Motivation . . . 1
- 1.3 Goals . . . 2
- 1.4 Block Diagram . . . 2
  - 1.4.1 System Changes . . . 3
- 1.5 IP Overview . . . 4
  - 1.5.1 Audio Processing . . . 4
  - 1.5.2 Dino Game . . . 4
  - 1.5.3 MicroBlaze Control . . . 5
- 1.6 Project Complexity . . . 5

2 Outcome **5**
- 2.1 Video Demo Link . . . 5
- 2.2 Results . . . 5
- 2.3 Future Work . . . 7
  - 2.3.1 Integrate Audio Response . . . 7
  - 2.3.2 Increase Game Complexity . . . 8

3 Project Schedule **8**
- 3.1 Milestones . . . 8
- 3.2 Discussion . . . 11

4 System Design Overview **11**
- 4.1 Audio Processing Unit . . . 11
  - 4.1.1 Audio Processing (Microphone Input to Audio Output) . . . 11
  - 4.1.2 Audio Score Converter . . . 12
- 4.2 Dino Game Unit . . . 15
  - 4.2.1 Custom IP: Game Control FSM . . . 15
  - 4.2.2 3rd Party IP: VGA Display . . . 15
  - 4.2.3 3rd Party IPs: Background/Dino/Obstacles Visuals . . . 16
- 4.3 MicroBlaze Control Unit . . . 16

5 Design Tree **17**

6 Tips and Tricks **18**

List of Figures

1 Dinosaur Game GUI 1
2 Block Diagram 2
3 Initial Block Diagram 3
4 Hysteresis Loop in Audio Score Converter IP 14
5 Dino Game FSM Diagram 15
6 Design Tree 18

List of Tables

1 Project Complexity Scores. 5
2 Description of the weekly milestones 8

1 Overview

ShoutMaster3000 is an audio-controlled dinosaur game using Nexys 4 DDR board inspired by Google's dinosaur game. The player's onboard microphone input is captured and fed into the dinosaur game. This audio signal controls the dinosaur's jumps, allowing it to hop over obstacles. The VGA displays the game (without MicroBlaze involvement). Additionally, the mono audio port on the FPGA outputs the processed audio, serving as a confirmation that input sounds are being faithfully captured and processed. Finally, a score tracker implemented using the MicroBlaze and AXI interface sends the score to the SDK terminal.

1.1 Background

Google's dinosaur game is a browser-based game in which the player guides a pixelated dinosaur through a side-scrolling landscape as shown in Fig. 1, avoiding obstacles to achieve a high score. The dinosaur dies when it collides with obstacles such as cacti and birds. Our goal is to use a processed audio signal to control the dinosaur's movement, rather than relying on the keyboard, buttons, or mouse found in the original game, and to display the game GUI on a VGA display.

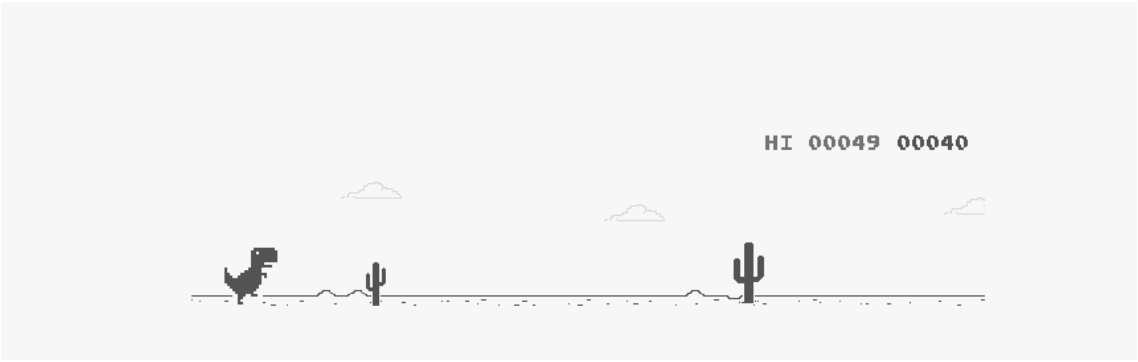

Figure 1: Dinosaur Game GUI

1.2 Motivation

Our group was inspired by TikTok's *Perfect Pitch Challenge*, where players must sing or produce sounds at specific frequencies to interact with the game. We saw an opportunity to blend that idea with hardware design by replacing traditional input methods – such as keyboards or buttons – with voice.

Additionally, we were eager to explore audio processing on FPGAs and understand how sound signals could be used as reliable control inputs in real-time systems. By building an audio-controlled version of Google's dinosaur game, we aimed to create an interactive and engaging experience that showcased both creative gameplay and practical signal processing on hardware.

1.3 Goals

The primary goal of this project is to develop an audio-controlled dinosaur game. To accomplish this, we aim to meet the following objectives:

- **Clear sound capture:** We will use a mono audio output to verify the accuracy of the captured sound. Ensuring clear sound capture is crucial because it guarantees that the audio input is correctly recorded, forming the foundation for precise audio processing and responsive game control.
- **Audio processing:** The system will convert a stream of binary data (0s and 1s) into a square wave and then accumulate this wave to create a volume-sensitive button. This step is essential as it transforms raw audio signals into a reliable control input, allowing the game to interpret the player's vocal commands effectively.
- **Real-time visualization:** The game will be displayed on a VGA monitor without using MicroBlaze, which helps reduce latency. Minimizing latency is vital for ensuring a smooth and immersive gaming experience, as it allows the game to respond to audio inputs in real time.

1.4 Block Diagram

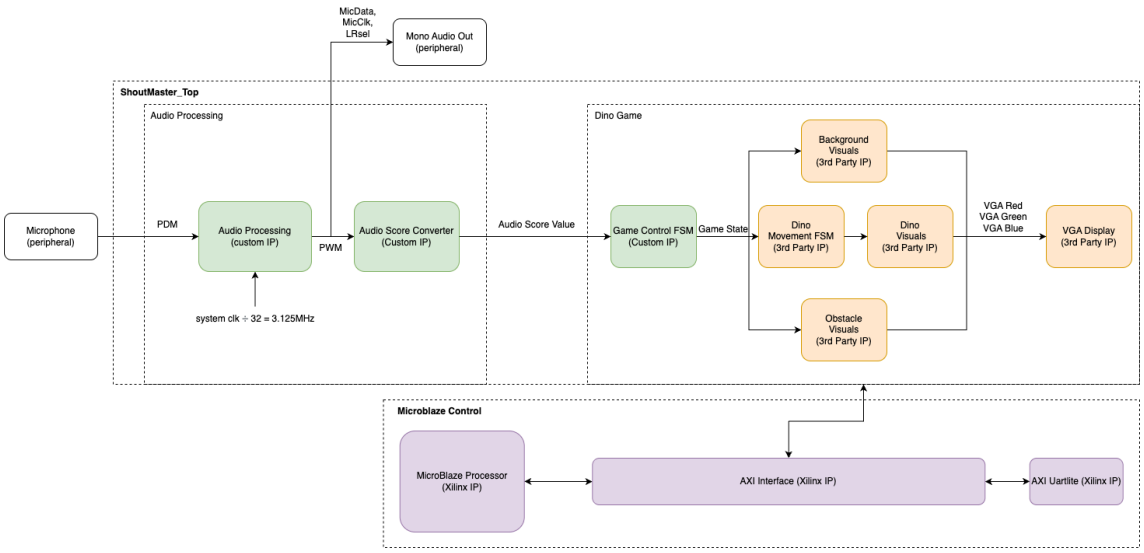

Figure 2: Audio Controlled Dino Game System Block Diagram

As shown in Fig. 2. This block diagram outlines the system architecture, divided into three main sections: Audio Processing, Dino Game, and MicroBlaze Control.

Audio input is captured by a microphone and processed through two custom IP blocks in the Audio Processing section. The first module converts high-speed PDM audio into a PWM signal, which is then translated into an Audio Score Value by the Audio Score Converter.

The Dino Game section, located centrally, includes custom and third-party IPs responsible for game logic and VGA output. The Game Control FSM uses the Audio Score Value to generate a jump command, which is interpreted by the Dino Movement FSM to control the dino's behavior. Visual elements—background, obstacles, and the dino—are rendered in real time via third-party IPs that output VGA signals to the display.

The MicroBlaze Control section, at the bottom, facilitates real-time communication via an AXI interface. It receives obstacle avoidance data from the Obstacle Visuals module, then displays this data through UART protocol to the user.

1.4.1 System Changes

In the initial block diagram depicted in Fig. 3, our design aimed to establish a client-server connection via Ethernet. In this configuration, the Nexys 4 DDR board acted as the client, sending audio data to a laptop server. This server then activated a Python application that displayed the dinosaur game on the laptop.

However, to achieve lower latency and enhance the responsiveness of the game, we revised our approach. We eliminated the Ethernet connection and integrated a direct VGA display into the system. By doing so, the game bypasses network transmission delays, allowing for faster processing of audio inputs and real-time visualization. This streamlined architecture not only simplifies the overall design but also ensures a smoother and more immediate gaming experience.

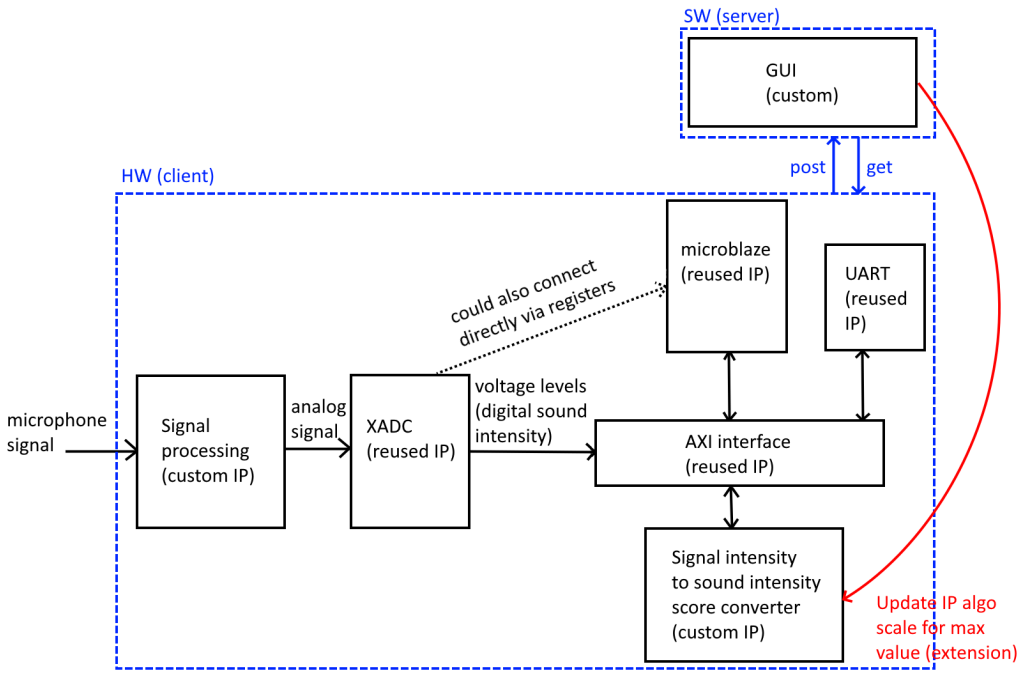

Figure 3: Audio Controlled Dino Game System Initial Block Diagram

1.5 IP Overview

1.5.1 Audio Processing

The Audio Processing module is integral to the system, serving two main functions: generating a mono audio output from the onboard microphone and providing a control signal for the dinosaur's movement. This dual functionality ensures both high-quality audio output and responsive, audio-controlled gameplay.

Audio Processing Custom IP

Firstly, the audio processing custom IP implements a PDM-PCM-PWM conversion pipeline. In this process, the Pulse-Density Modulation (PDM) stream from the microphone is sampled through Pulse-Code Modulation (PCM) and then directly converted to a Pulse-Width Modulation (PWM) signal for output. These are different audio formats which are needed based on the spec for the microphone and mono audio port. This conversion results in a clean audio output signal that can be played through a speaker.

Audio Score Converter Custom IP

Secondly, the audio score converter custom IP accumulates the PWM signal output to derive a numerical value that serves as an input to the dinosaur module. By comparing the accumulated value against a set threshold, the system determines when to evoke the dinosaur's movement. If the audio intensity, as measured by the aggregated PWM signal, exceeds this threshold, it triggers the dinosaur to jump. This results in a dinosaur game control that is sensitive to the player making sound, but is not overwhelmed by the ambient noise level.

1.5.2 Dino Game

The Dino Game module mimics the game mechanism, graphics, and rules of Google's popular dinosaur game. This IP leverages a processed audio signal as the game input, allowing sound levels to trigger game events, and outputs a 640x480 video display through the VGA port—all without using a Microblaze processor.

The game features a custom finite state machine (FSM) that governs the dinosaur's behavior, transitioning through states such as initialization, active play, and game over. The dinosaur initiates a jump when the processed audio signal exceeds a predefined threshold, effectively replacing traditional button-based inputs with an innovative, audio-driven control mechanism. The VGA controller is designed to render smooth graphics and animations at a resolution of 640x480, faithfully reproducing the retro aesthetic of the original game.

Furthermore, the system includes modules for obstacle generation, collision detection, and background visuals. Obstacles appear at varying intervals and speeds to challenge the player, while collision detection logic seamlessly integrates with the FSM to transition the dinosaur into the 'dead' state upon impact.

Overall, this IP exemplifies a resource-efficient design tailored for FPGA implementation. By eschewing complex microprocessor cores in favor of streamlined, dedicated hardware logic, the Dino Game IP achieves both real-time performance and a unique, audio-driven gameplay experience.

1.5.3 MicroBlaze Control

The MicroBlaze Control module serves as the system's score tracker. It works closely with the Dino Game Module by monitoring obstacle detection events, such as successful jumps to avoid obstacles. Each time the dinosaur performs a jump to evade an obstacle, the module increments a jump counter and stores this updated value in an AXI slave register.

The MicroBlaze processor then reads the score from the AXI register via the AXI bus. Once retrieved, the score is transmitted to the SDK terminal, allowing for real-time display of the player's performance. This setup not only ensures efficient tracking of gameplay events but also offloads the score monitoring task from the main game processing, contributing to a smoother and more responsive gaming experience.

1.6 Project Complexity

Our project complexity scores are calculated according to Table 1.

Table 1: Project Complexity Scores.

Component	Complexity Score
Onboard Microphone	0.5
Onboard Audio Output Port	0.5
VGA Display Without Microblaze	1
GUI	0.75
Custom IP Cores Implemented in HW	0.75
Total	3.5

2 Outcome

2.1 Video Demo Link

The video demo can be watched via Google Drive Link.

2.2 Results

Our project successfully met the core objective: creating a functional audio-controlled dinosaur game using the Nexys 4 DDR board. While the path to completion involved

some unexpected pivots, the final system reliably captures sound input, processes it, and uses it to control the dinosaur through a volume-sensitive mechanism.

Audio Processing

Initially, we planned to incorporate advanced filters from the Xilinx IP suite to clean the microphone input. However, we encountered integration challenges with those IPs and realized that even without them, the raw PDM signal—after conversion to PCM and PWM—was sufficiently clean for our purposes. This allowed us to simplify the audio processing pipeline, focusing instead on basic conversion and signal shaping. This pivot not only saved development time but also reduced latency and complexity. The mono audio output confirmed the clarity of captured sound, providing both a debugging tool and user feedback.

Audio Score Converter

Contrary to our expectations, designing the audio score logic proved more difficult than the audio processing itself. The challenge lay in distinguishing user-generated sound from background noise in a way that was both responsive and robust. For example, sharp background spikes could mimic a user shout, and tuning the system to suppress these false positives while still recognizing valid input required extensive testing and iteration.

We addressed this using a combination of PWM accumulation, decay, and hysteresis. While the final version functioned well in most environments, it remained sensitive to edge cases, such as overlapping background noise levels with player input. Ultimately, the system worked as intended but required careful calibration to strike a balance between noise resilience and responsiveness. This highlighted the difficulty of implementing simple yet reliable audio-based control in real-world settings.

Dino Game

The Dino Game IP was one of the highlights of the project, transforming real-time audio input into game control using a fully hardware-accelerated architecture. At its core, the game was managed by a custom finite state machine (FSM) that transitioned through three states: idle, alive, and dead. The FSM monitored the audio score signal and triggered a jump action when the volume exceeded a preset threshold.

One of the key challenges we faced in this module was integrating our custom FSM with third-party visual IPs for rendering the dinosaur, background, and obstacles. These visuals were generated using external scripts that converted pixel data into Verilog assignment logic. While this allowed for efficient rendering and a clean VGA output, it introduced complications in timing and coordination with our FSM.

For instance, we had to ensure that the jump state in our FSM correctly triggered and synchronized with the animation of the dinosaur on-screen, while also aligning

with the movement of obstacles and collision detection logic. Since the visual modules were pre-generated and not inherently state-aware, we had to carefully map our control signals to their rendering conditions. Mismatches in timing or positioning caused visual glitches and incorrect collision detection.

Despite these challenges, the final implementation worked reliably. The dinosaur responded immediately to valid audio input, obstacles moved across the screen at consistent intervals, and the FSM managed state transitions without lag. The combination of custom logic and visual modules resulted in a responsive and engaging user experience, validating our decision to offload gameplay mechanics to dedicated hardware rather than using a soft processor like MicroBlaze.

Final Feature Comparison

- **Clear sound capture:** The mono audio output is clean, and this was achieved with a simpler implementation than expected.
- **Audio processing:** Implemented a functional volume-based control, though tuning the scoring thresholds proved more involved than anticipated. This module was completed without Xilinx filters, preserving signal clarity with minimal processing.
- **Real-time visualization:** Gameplay was achieved using VGA output with minimal latency, bypassing Microblaze. The game runs smoothly and the user experience is good.

Overall, the system demonstrated that a minimal design philosophy can outperform over-engineered solutions, especially when timing, responsiveness, and robustness are priorities. If we were to start over, we would likely begin with a minimal setup and build up only as necessary, rather than trying to integrate complex third-party tools from the outset. For future development, we recommend expanding the game's visual complexity and obstacle variety, and improving the sensitivity tuning of the audio scoring logic. Additionally, integrating real-time audio visualizations (e.g., time or frequency response) onto the VGA display would provide clearer user feedback and enhance the interactivity of the system.

2.3 Future Work

2.3.1 Integrate Audio Response

Currently, users only hear the mono audio output, which limits the visual understanding of how their sound input is processed. To enhance the user experience, future work could involve integrating both time-domain and frequency-domain visualizations onto the VGA display directly, which would provide real-time feedback on the audio input. A waveform could be rendered in real-time to represent the amplitude variations of the incoming audio signal, while a frequency-domain visualization – implemented using a real-time Fast Fourier Transform (FFT) – would illustrate the dominant frequency components of the audio. These complementary

visualizations could provide insight into the audio processing pipeline, allowing users to see how variations in their input (such as shouting) affect the signal.

2.3.2 Increase Game Complexity

Due to time constraints, the current implementation features only a single type of obstacle: a cactus. Future iterations of the game could expand the obstacle repertoire to include multiple obstacle types, such as various sizes of cacti, birds that appear at different heights, and other environmental challenges. This diversity will contribute to a more dynamic and engaging gameplay experience. Furthermore, we could introduce a control interface – such as a dedicated button –that enables players to switch between different game speeds (e.g., fast and slow modes). Such enhancements aim would not only diversify the gaming challenges but also provide a more customizable experience for users.

3 Project Schedule

3.1 Milestones

A summary of our original and completed milestones can be seen below.

Table 2: Description of the weekly milestones

Original Milestones	Final Milestones
Milestone 1	
Software: • Set up basic UART communication between the FPGA and PC. • Develop a basic GUI prototype to visualize test values.	Software: • Found open-sourced library - Phaser • Finished 3/4 of the first tutorial (Environment Deployment)
Hardware: • Setup XADC: develop testbench, inject synthetic signals, Verify output	Hardware: • Researched signal processing path (microphone to XADC) and realized XADC was unneeded • Started research into signal processing IP

Continued on next page

Table 2 – *Continued from previous page*

Original Milestones	Final Milestones
Milestone 2	
Software: • Develop a GUI interface with a dynamic real-time intensity meter (mock values initially). • Implement real-time UART data reception into the GUI. Hardware: • Implement IP for microphone sound capture • Set up testbench for signal processing IP	Software: • Ran multiple examples in Phaser • Built a Dinosaur T-Rex in PyGame (more audio-related examples) • Designed a basic UI (collecting asset images) Hardware: • Set up on-board microphone data capture, working • Developed testbench for microphone, waveform validated
Milestone 3	
Software: • Integrate UART data stream into the GUI • Ensure the GUI dynamically updates sound intensity values in real time (check under 50ms latency) Hardware: • Implement IPs for microphone signal processing (e.g., filtering) • Set up testbench for signal processing IP	Skipped, no progress made.
Milestone 4	

Continued on next page

Table 2 – *Continued from previous page*

Original Milestones	Final Milestones
Software: • Fine-tune real-time GUI visualization based on test data • Debug latency issues and ensure smooth user interaction Hardware: • Implement audio-to-score converter IP • Implement dino game state controls	Software: • Port game visuals from SW to VGA display • Determined pixel values for each figure Hardware: • Scrapped integrating Xilinx IP for high/band/low-pass filters for microphone data; wrote custom IP for outputting sound to mono audio out port instead • Wrote audio score converter IP, but occasional flickering under debug • Wrote FSM for dino game state, will integrate for testing next milestone
Milestone 5	
• Implement Microblaze to track game score • Debug audio score converter IP • Integrate sound acquisition and signal processing IP with dino game VGA display	• Microblaze design implemented, but game score is incorrect, still under debug • Audio score converter IP optimized for performance (response speed, stability, and filtering quality) • Linked microphone signal capture and processing IP to VGA display; dino responds to audio
Milestone 6	

Continued on next page

Table 2 – *Continued from previous page*

Original Milestones	Final Milestones
• Fully integrate sound acquisition, IP block processing, Microblaze UART transmission, and VGA GUI visualization • Test system to simulate real-world arcade gameplay	• Microblaze score tracker fixed and integrated with dino game • Dino game speed and obstacle buffer size reduced to improve gameplay experience

3.2 Discussion

Our actual progress ended up being quite different from what we originally planned. In the early stages, we realized that some of our initial ideas – like using XADC for signal capture – weren't the best fit for our project. So we changed direction and started looking into other solutions, such as signal processing IPs. On the software side, we found open-source tools and libraries, which helped speed up our GUI development. Because of these changes and the time spent adjusting, we had to skip Milestone 3. As we moved forward, we spent more time than expected debugging, especially with the audio score converter IP and the Microblaze game score system, which didn't work perfectly at first.

Still, we made solid progress by the end. We managed to get the full system working, with the game responding to audio input and improvements made to both gameplay and performance. Even though the schedule didn't go exactly as planned, we adapted well and focused on the most important parts to make sure the project was successful.

4 System Design Overview

4.1 Audio Processing Unit

The Audio Processing module is integral to the system, serving two main functions: (1) generating a clear mono audio output from the onboard microphone and (2) providing a control signal for the dinosaur's movement.

4.1.1 Audio Processing (Microphone Input to Audio Output)

The audio processing custom IP handles the conversion of the microphone input to the audio output through a pipeline of digital signal formats. The entire process can be divided into three main stages: PDM input, PCM conversion, and PWM output.

1. **PDM input (from the microphone)**

- Format: Pulse-Density Modulation (PDM)
- Description: The microphone provides a digital stream of 0s and 1s. The density of 1s corresponds to the amplitude of the audio signal.
- Implementation: The PDM signal is clocked using the microphone clock, which is between 2-4 MHz as per the spec [1]:

$$\text{mic\_clk} = \frac{\text{sys\_clk}}{32} = 3.125\,\text{MHz}$$

2. **PCM conversion**
    - Format: Pulse-Code Modulation (PCM)
    - Description: The PDM stream is sampled to obtain the "code" for PCM. The sampled value corresponds to the amplitude of the audio signal at a given moment. A higher value means higher amplitude.
    - Purpose: Convert the high-frequency binary PDM stream into a discrete, multi-bit digital representation for further processing.
    - Implementation: The system samples the input at `mic_clk`.
3. **PWM output (to the mono audio port)**
    - Format: Pulse-Width Modulation (PWM)
    - Description: PWM is a square wave whose duty cycle is proportional to the amplitude.
    - Implementation: Each PCM sample sets the PWM output high or low depending on the amplitude transition.

Naturally, a question arises: "Why do we need PCM?" If PDM is the format needed for the input and PWM is that needed for the output, why not convert from PDM to PWM directly? It seems like the middle PCM stage is unnecessary.

Actually, the PCM stage arises not out of necessity but through implementation. By its definition, PCM is sampling of the PDM data stream. This can be complex (such as accumulating and averaging over several cycles) or crude (such as just sampling a bit at a single point in time). Either way, because the data must be sampled for any further processing downstream, PCM is performed.

4.1.2 Audio Score Converter

The audio score converter custom IP implements a volume-sensitive button mechanism using PWM audio input. It converts audio activity into a numerical score that serves as an input to the dinosaur module. By comparing this value against a set threshold, the system determines when to evoke the dinosaur's movement.

Essentially, the audio score converter is a volume-sensitive button. The idea is to use audio volume, represented by the duty cycle of a PWM signal accumulated over

time, to "press" a virtual button. Louder sounds push the internal score higher, while quieter periods let it decay.

To make the system robust against noise and saturation and ensure responsive gameplay, the score converter was designed using three core mechanisms: PWM accumulation, decay, and hysteresis.

1. **PWM accumulation**
   On every clock cycle, if the PWM input is high, a constant STEP value is added to an internal counter representing the current sound level. Over time, sustained high-PWM signals (i.e., louder sounds) cause the counter to grow. This simulates audio energy accumulation.

   This was needed in the implementation because the PWM signal, a square wave, cannot be used directly to compare to a threshold.

2. **Decay**
   At the same time, on each cycle, a baseline amount is subtracted from the internal counter. This simulates the natural decay of sound energy over time. It prevents saturation of the counter when the input is consistently high and allows the score to fall back down when the input becomes quiet. It also allows the system to be tuned for a background noise level, helping filter out player-created noise from ambient noise.

3. **Hysteresis**
   To make the score-to-button logic robust, the system introduces hysteresis, as seen in Fig. 4, which illustrates the hysteresis loop, showing different paths for activation and deactivation based on the internal score level.

   Two thresholds are used:

   - THRESHOLD_ON: The counter (accumulated PWM signal with decay) must rise above this level to trigger a "button press."
   - THRESHOLD_OFF: The counter must fall below this level to reset or "unpress" the button.

   This prevents flickering (rapid toggling) due to noise or small fluctuations in the PWM signal, which could be triggered by sudden spikes in background noise.

   Highlighting key points in the hysteresis loop in Fig. 4:

   - Point D & C: when jump=0 (a jump is not detected), if the counter level is below THRESHOLD_ON, jump=0 still
   - Point B: when the counter level exceeds THRESHOLD_ON, jump=1 (a jump is detected)
   - Point A & F: when jump=1, until the counter level falls below THRESHOLD_OFF, jump=1 still (note this implies that if the counter level falls below THRESHOLD_ON but remains above THRESHOLD_OFF, the dinosaur will still be considered as jumping)

- Point E & F: when jump=1, only if the counter level falls below THRESHOLD_OFF does jump transition low

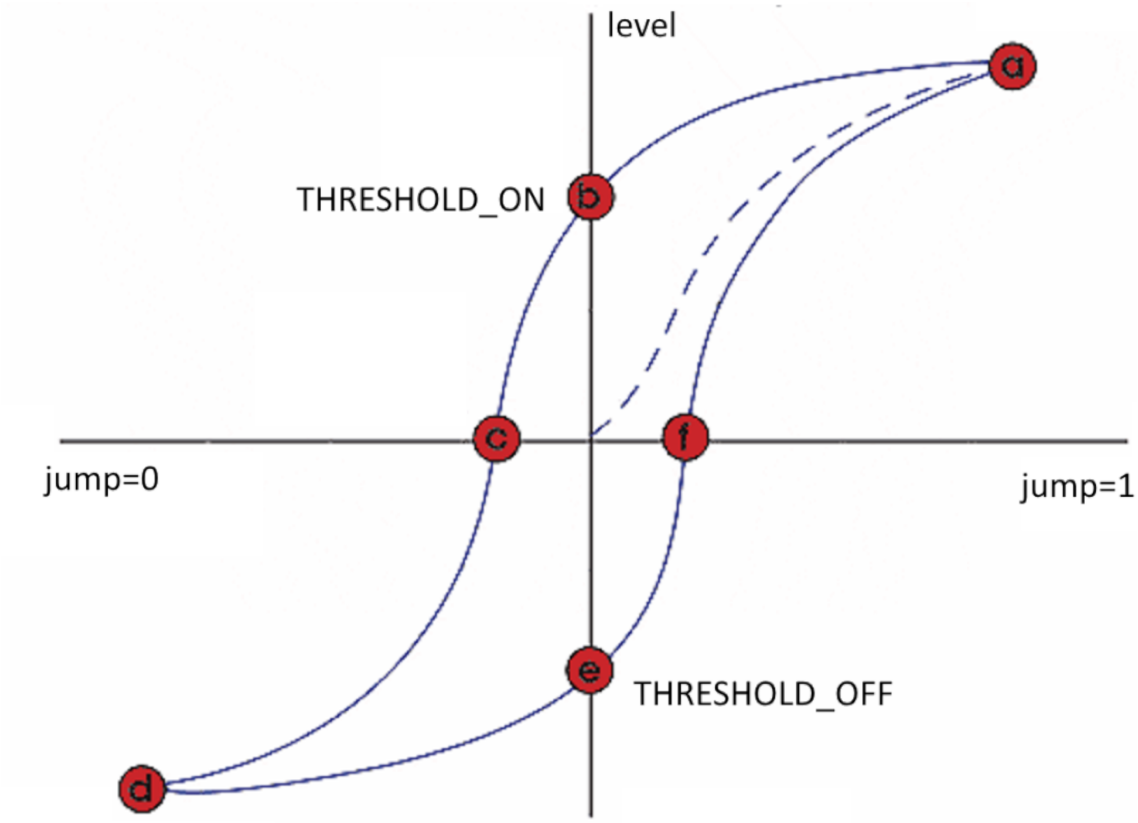

Figure 4: Hysteresis Loop in Audio Score Converter IP

In terms of testing, to determine THRESHOLD_OFF and THRESHOLD_ON, we first compiled the hardware with the audio score converter counter signal connected to the LEDs. We tested this system under ambient-level and shout-level noise, and noted the counter values displayed on the LEDs. This helped us obtain a baseline value for the expected noise levels.

This process was an iterative one, as in the first few designs, we noticed that the counter signal did not have high enough precision – the ambient noise and shouts resulted in similar converted values. To address this, we increased the size of the counter signal. However, we needed to balance this with increasing the STEP size, to ensure the system would still respond quickly to noise level changes while at higher precision.

The scores for ambient-level and shout-level noise were used as a guideline for the THRESHOLD_OFF and THRESHOLD_ON values respectively. Then, after the threshold values were established, the baseline decay value was determined by binary search testing.

4.2 Dino Game Unit

4.2.1 Custom IP: Game Control FSM

As shown in Fig. 5, the Dino Game is managed by a finite state machine comprising three states: initial, alive, and dead. In the initial state (S0), the dinosaur waits for the audio score value from the Audio Processing Module. Once the audio output reaches the required threshold, the dinosaur transitions to the alive state (S1) and begins jumping. While in the alive state, collision detection runs periodically. If a collision is detected, the dinosaur transitions to the dead state (S2); if no collision occurs, it continues jumping and remains in the alive state.

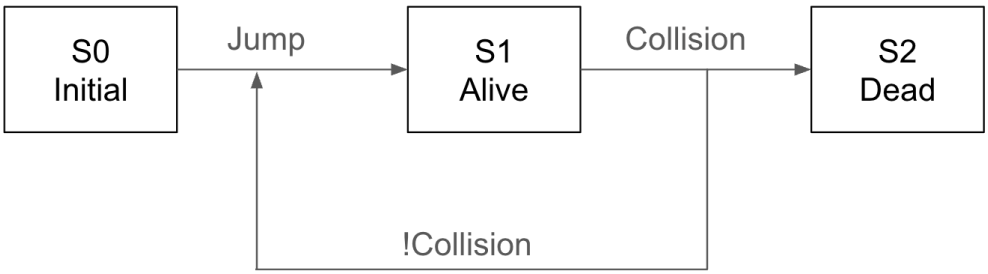

Figure 5: Dino Game FSM Control Flow Diagram

Detection Collision

The collision detection mechanism operates across three distinct timestamps:

- **At time = 0:** The system checks for any overlap between the dinosaur and an obstacle. The finite state machine (FSM) maintains pointers to both objects and uses an AND gate to detect overlapping.
- **At time = 1:** If an overlap between the cactus and the dinosaur is detected, the FSM holds for one time period. This pause is designed to reduce the buffer between the two; without this hold, the distance would be too great, negatively affecting the gaming experience.
- **At time = 2:** The dinosaur transitions to the final dead state (S3).

4.2.2 3rd Party IP: VGA Display

To draw the pixel values on the VGA display, a python script is used to generate the corresponding Verilog code. This script scans through image files and converts the pixel data into assignment statements and module definitions that can be directly integrated into the FPGA design. Below is an explanation of how the script operates:

1. **Image Analysis and Parameter Extraction**
   The script uses the Python Imaging Library (PIL) to open each image file and retrieve its dimensions and pixel data. Functions like AnalyzerInit and AnalyzeImage extract the width, height, and pixel values of the image. These

dimensions are then converted into Verilog parameters (e.g., object width and height defined in terms of pixel size "px") to ensure that the drawn object is scaled correctly on the VGA display.

2. **Pixel Grouping and Assignment Generation**
   The core function, Analyzer, processes the image by scanning each pixel. It looks for specific color values (such as BLANK, GREY, and CLOUD_GREY) that represent different parts of the object. The script groups consecutive pixels into horizontal line segments using Python's `itertools.groupby` and `operator` modules. These segments are then used to create Boolean expressions that determine whether a given VGA coordinate (X, Y) falls within the object's "hitbox." For example, the script generates assignments like:

```
assign inGrey_object = ((X > ox + offset_X1) \\
&& (X <= ox + offset_X2) && (Y == oy + offset_Y1)) \\
|| ... ;
```

4.2.3 3rd Party IPs: Background/Dino/Obstacles Visuals

The game creates the illusion of forward motion by coordinating the movement of different elements. The dinosaur appears to move from left to right while the obstacles and background scroll from right to left, resulting in a continuous, immersive environment.

For the dinosaur, a pointer tracks its right side. When this pointer reaches the right boundary of the display, it is reset to the left boundary, making the dinosaur reappear from the left. Otherwise, the pointer increments by 1 to move the dinosaur further to the right.

For the obstacles and the background, a pointer is maintained for their left side. When this pointer hits the left boundary, it is reset to the right boundary, allowing these elements to re-enter the scene from the right. If the pointer has not yet reached the left boundary, it decrements by 1, moving the elements to the left.

4.3 MicroBlaze Control Unit

The MicroBlaze Control Unit comprises several key components, including the MicroBlaze Processor, AXI Interconnect, AXI Uartlite, and the custom ShoutMaster3000 AXI Interface Module. This unit functions primarily as the system's score tracker, operating in tandem with the Dino Game Module to monitor obstacle avoidance events—specifically, successful jumps over obstacles.

A custom module within the game logic detects each successful jump and increments a counter, which is stored in an AXI slave register. The MicroBlaze processor continuously polls this register via the AXI interface to retrieve the current score. Once obtained, the score is transmitted through the AXI Uartlite to the SDK terminal

for real-time performance monitoring.

While the current implementation focuses on score tracking and display, this architecture lays the groundwork for future enhancements. For example, the score could be used to dynamically adjust game difficulty or be stored for high score tracking—features that could further enrich the gaming experience in future iterations.

5 Design Tree

The Github repository can be found here: Github repository.

The design tree can be seen in Fig. 6.

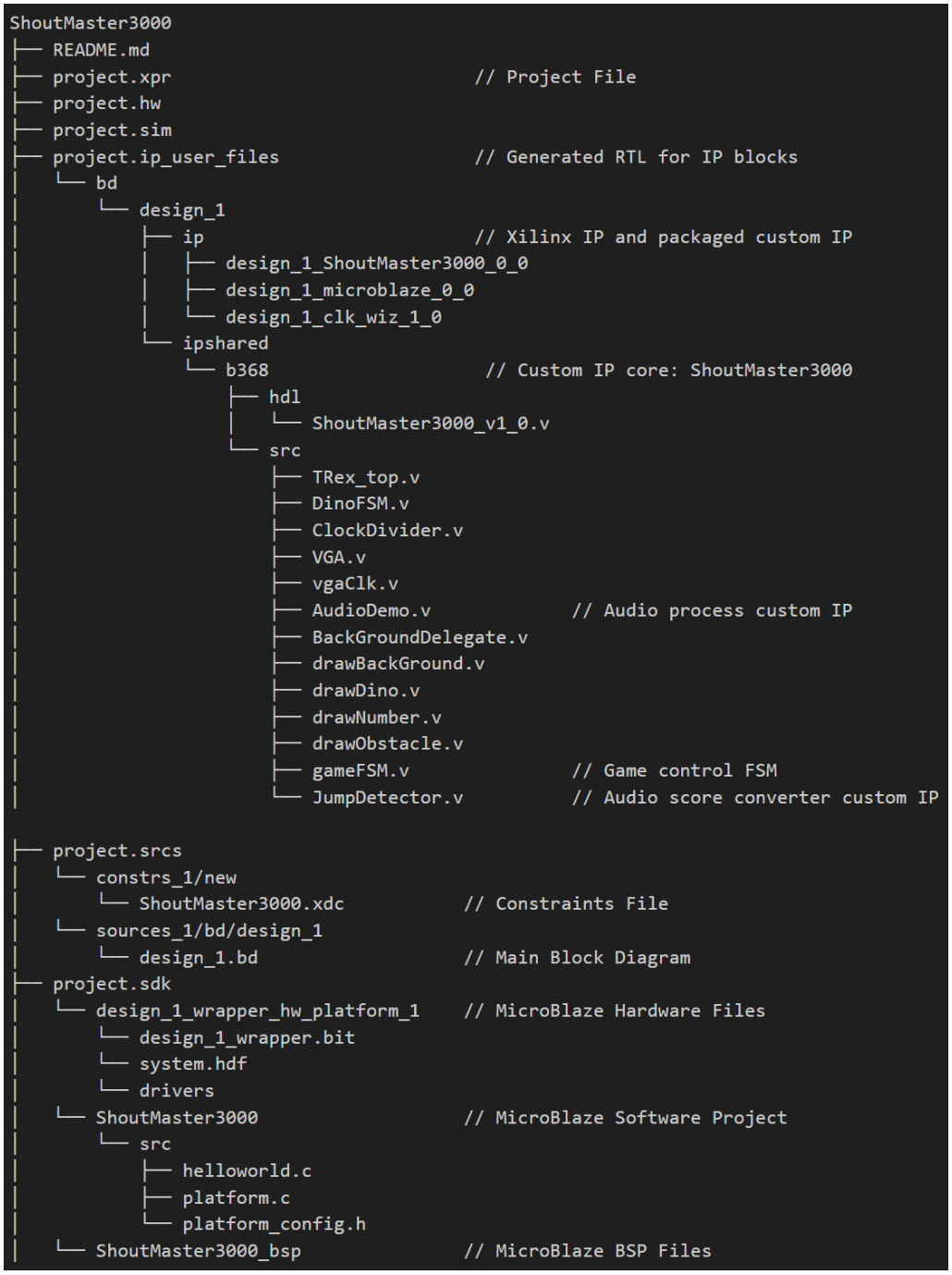

Figure 6: Design tree

6 Tips and Tricks

- **Start Simple, Then Scale**
  It's tempting to begin with advanced IP blocks and complex subsystems, but we found it far more productive to first get a basic version working. Our audio processing pipeline worked surprisingly well without complex Xilinx filters.

- **Use Intermediate Debugging Checkpoints**

Checking intermediate signals (like the audio score counter) by connecting them to the on-board LEDs helped us visualize system behavior in real time, even though it was not the hardware intended for our final design. This was especially useful during parameter tuning and calibration.

- **Don't Rely Too Heavily on Third-Party IPs**
  Many third-party IPs are not well-documented or are overly rigid. In our case, integrating pre-generated visual modules into our FSM required a lot of effort. If you use external modules, make sure you fully understand their internal logic and timing.

- **Use Simulation Before Hardware**
  Even basic waveform simulations saved us from many potential bugs before deploying to the board. It's much faster to debug in ModelSim than re-synthesizing every change.

- **Leave Time for VGA Rendering**
  Generating and aligning visuals with control logic took more time than we expected, especially with timing-sensitive modules like FSMs and collision detection. Budget time accordingly, and test each layer (game logic, visuals, movement) independently.

References

[1] Inc. Digilent. Nexys 4 ddr reference manual, 2024. URL https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual. Accessed: 2025-04-06.